Lab 02 - Ridge Regression

[Your Name Here]

Due 26 September at 11pm

Brief description

In this lab, we will perform ridge regression on the prostate cancer data set studied last time. Recall that the 9th variable, `lpsa`, is the response while the rest are potential predictors. the variable `lpsa` in the 9th column is the response while the rest are potential predictors. The data set is available in the `{ElemStatLearn}` package and a description can be found at http://statweb.stanford.edu/~tibs/ElemStatLearn/datasets/prostate.info.txt

Questions

Like last time, read the data into R using the following command (with correct file path and name):

```
data(prostate, package = "ElemStatLearn")
library(tidyverse)
prostate <- prostate |> select(-train)
```

Q1

What are the coefficients for the predictors `lcavol`, `svi` and `lcp` in the full linear regression model? (If you recall, these are the three predictors most correlated with the response.)

```
# Fit the full model with all predictors
fit_full <- lm(lpsa ~ ., data = prostate)

# Look at the coefficients
coef(fit_full)[c("lcavol", "svi", "lcp")]
```

```
##     lcavol        svi        lcp
##  0.5643413  0.7616734 -0.1060509
```

```
summary(fit_full)
```

```
##
## Call:
## lm(formula = lpsa ~ ., data = prostate)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.76644 -0.35510 -0.00328  0.38087  1.55770
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  0.181561   1.320568   0.137  0.89096
## lcavol       0.564341   0.087833   6.425 6.55e-09 ***
## lweight      0.622020   0.200897   3.096  0.00263 **
## age         -0.021248   0.011084  -1.917  0.05848 .
## lbph         0.096713   0.057913   1.670  0.09848 .
## svi          0.761673   0.241176   3.158  0.00218 **
## lcp         -0.106051   0.089868  -1.180  0.24115
## gleason      0.049228   0.155341   0.317  0.75207
## pgg45        0.004458   0.004365   1.021  0.31000
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6995 on 88 degrees of freedom
## Multiple R-squared:  0.6634, Adjusted R-squared:  0.6328
## F-statistic: 21.68 on 8 and 88 DF,  p-value: < 2.2e-16
```

For responses $y_1, \ldots, y_n$, vector of predictors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ and fixed penalization parameter λ, ridge regression solves the optimization problem

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left[\sum_{i=1}^{n} (y_i - \boldsymbol{x}_i^\intercal \boldsymbol{\beta})^2 + \lambda \boldsymbol{\beta}^\intercal \boldsymbol{\beta} \right],$$

where $\boldsymbol{\beta}$ is the parameter vector (regression coefficients) and λ is a tuning parameter that penalizes $\boldsymbol{\beta}$'s that are "too large". It can be shown that the solution to ridge regression is

$$\hat{\boldsymbol{\beta}} = (\boldsymbol{X}^\intercal \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}^\intercal \boldsymbol{y},$$

where $\boldsymbol{X}$ is the design matrix, $\boldsymbol{y}$ is the response vector and $\boldsymbol{I}$ is the identity matrix of appropriate size.

Q2

What is the value of λ for the regression considered in **Q1**? As $\lambda \to \infty$, what is the behaviour of the vector $\hat{\boldsymbol{\beta}}$ (the ridge regression estimator)?

Your answer

Now, we perform ridge regression using the function `glmnet` in the package of the same name.Load the package using `library(glmnet)`. Read the documentation of the function by typing `?glmnet` in the console. We are interested in these four arguments of the function: `x`, `y`, `alpha` and `lambda`.

We consider the sequence of penalties $\boldsymbol{\lambda} = (e^{-3}, e^{-2.8}, e^{-2.6}, \ldots, e^{2.6}, e^{2.8}, e^{3})$. Create this vector of length 31 and store it as variable `lam`. Fit the series of ridge regressions to the data set. You will need to use the following command:

```
fittedobject <- glmnet(x = ___, y = ___, alpha = 0, lambda = ___)
```

where `alpha = 0` specifies the ridge regression (as opposed to other shrinkage methods). Replace the `___` by the appropriate variable names.

Hint: The argument `x` accepts a matrix. You can use `as.matrix()` to convert a data frame to matrix. Or, combine a formula with `model.matrix()` like `model.matrix(y ~ var1 + var2, data = df)`. Try `?model.matrix`. Note that `model.matrix()` adds an intercept column, which `glmnet()` does as well, so you'll have to get rid of one if you go this route.

```r
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```r
# 1) Lambda sequence: e^{-3}, e^{-2.8}, ..., e^{2.8}, e^{3}  (length = 31)
lam <- exp(seq(-3, 3, by = 0.2))

# 2) Design matrix (no intercept column because glmnet adds it)
X <- model.matrix(lpsa ~ ., data = prostate)[, -1]

# 3) Response
y <- prostate$lpsa

# 4) Fit ridge regressions across the lambda grid
fittedobject <- glmnet(x = X, y = y, alpha = 0, lambda = lam)

# (Optional) peek
# fittedobject
coef(fittedobject, s = lam[1])     # coefficients at the first lambda
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##             s=0.04978707
## (Intercept)  0.062943832
## lcavol       0.518007868
## lweight      0.612330155
## age         -0.018363853
## lbph         0.089720487
## svi          0.712114235
## lcp         -0.062087242
## gleason      0.059477654
## pgg45        0.003728455
```

```r
coef(fittedobject, s = lam[length(lam)])  # at the last lambda
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##               s=20.08554
## (Intercept) 1.9286313089
## lcavol      0.0359260221
## lweight     0.0590038132
## age         0.0011243644
## lbph        0.0071556936
## svi         0.0770832831
## lcp         0.0214301081
## gleason     0.0270053939
## pgg45       0.0007959566
```

Q3

Notice that the fitted `glmnet` object is a list with many components. Inspect it by looking in the Environmet tab, printing it, calling `names(obj)` and / or trying `str(obj)`. Like most modelling routines in `R`, there is a `coef()` method for `glmnet` objects.

Write down the coefficients for the predictors `lcavol`, `svi` and `lcp` when $\lambda = e^3$ and $\lambda = e^0$, respectively.

Hint: `fittedobject` is a list. Variables in a list can be extracted using the `$` operator. For example, `fittedobject$lambda` gives you the λ values used in the series of ridge regressions.

```
# coefficients at lambda = exp(3)
coef_exp3 <- coef(fittedobject, s = exp(3))
coef_exp3[c("lcavol", "svi", "lcp"), ]
```

```
##     lcavol        svi        lcp
## 0.03592602 0.07708328 0.02143011
```

```
# coefficients at lambda = exp(0) = 1
coef_exp0 <- coef(fittedobject, s = exp(0))
coef_exp0[c("lcavol", "svi", "lcp"), ]
```

```
##     lcavol        svi        lcp
## 0.25869200 0.44539450 0.07611332
```

Next, we find the optimal value of λ (that yields the best predictive ability) via cross validation. The relevant function is `cv.glmnet()`. We will also need to use the `nfolds` argument in addition to those above (check the documentation of this function).

Q4

Carry out a 5-fold cross validation, using the same λ values as above. Run `set.seed(406)` **immediately before** the `cv.glmnet()` function to obtain reproducible results. Inspect the fitted object; which variable stores the cross validation mean squared errors?

```

```

```r
library(glmnet)

# same lambda grid as before
lam <- exp(seq(-3, 3, by = 0.2))
X <- model.matrix(lpsa ~ ., data = prostate)[, -1]
y <- prostate$lpsa

set.seed(406)
cvfit <- cv.glmnet(
  x = X, y = y,
  alpha = 0,
  lambda = lam,
  nfolds = 5,
  type.measure = "mse"    # explicit: MSE for gaussian
)

# The cross-validated mean squared errors (one per lambda):
cv_mse <- cvfit$cvm
head(cv_mse)
```

```
## [1] 1.1512407 1.1182379 1.0820311 1.0430026 1.0017361 0.9589988
```

Your answer

Q5

What is the value of λ that minimizes the CV error and what is its associated mean squared error?

Hint: You can visualize the result using `plot()` on the fitted object in **Q4**.

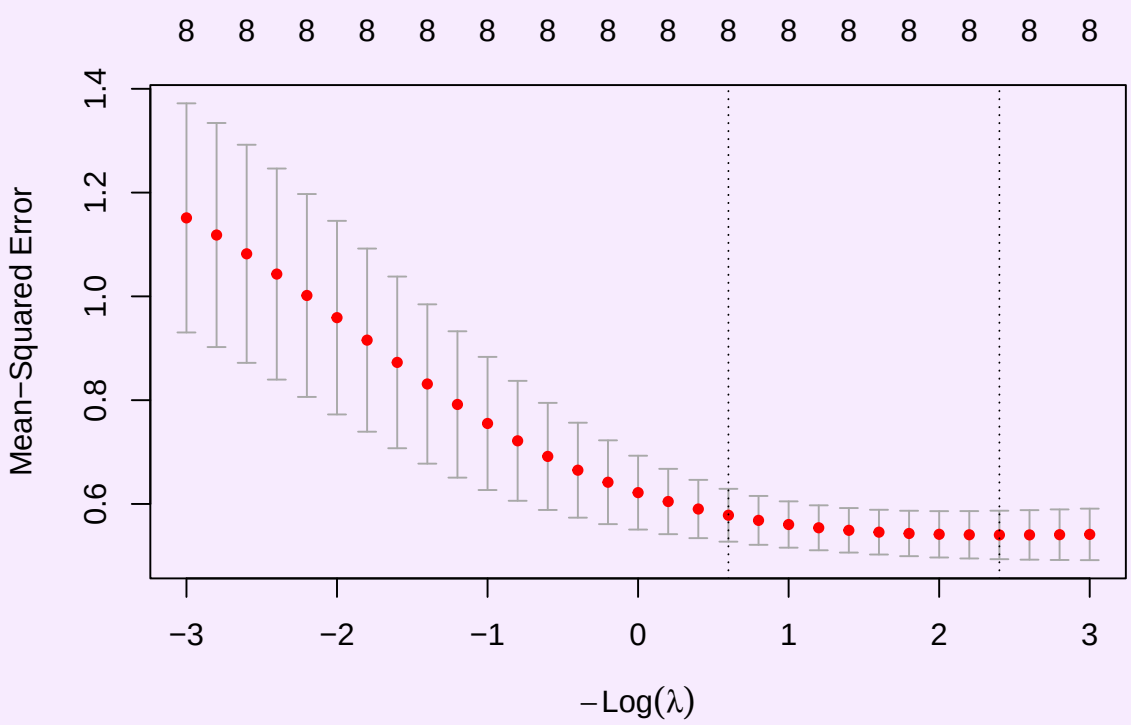
```r
# lambda that minimizes CV error
best_lambda <- cvfit$lambda.min

# associated CV mean squared error
best_mse <- min(cvfit$cvm)

best_lambda
```

```
## [1] 0.09071795
```

```r
best_mse
```

```
## [1] 0.5402706
```

```r
plot(cvfit)
```



Q6

For this (optimal) λ, what are the fitted coefficients for the predictors `lcavol`, `svi` and `lcp`?

```
# Coefficients at optimal lambda (lambda.min)
coef_opt <- coef(cvfit, s = cvfit$lambda.min)

# Extract lcavol, svi, and lcp
coef_opt[c("lcavol", "svi", "lcp"), ]
```

```
##      lcavol         svi         lcp
##  0.48791754  0.68115453 -0.03615836
```

```
coef(fittedobject, s = cvfit$lambda.min)[c("lcavol", "svi", "lcp"), ]
```

```
##      lcavol         svi         lcp
##  0.48791754  0.68115453 -0.03615836
```